

Memoria Dinámica

Antonio Espín Herranz

Contenidos

- Introducción
- malloc/calloc/realloc/free — patrones correctos y errores típicos.
- Memory leaks y dangling pointers — detección y prevención.
- Fragmentación y estrategias de asignación.
- Valgrind y sanitizers — análisis dinámico.

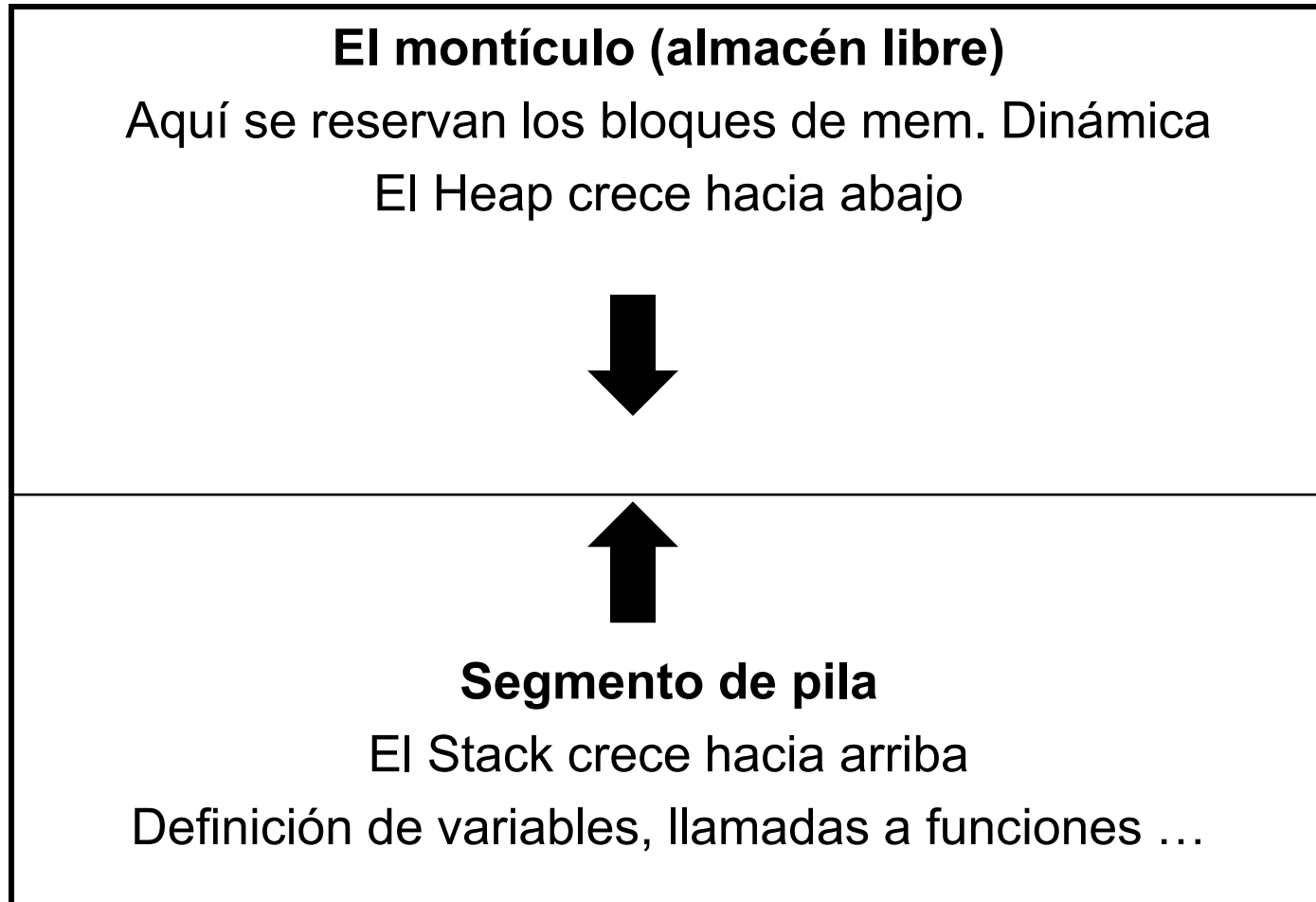
TRABAJAR CON MEMORIA DINÁMICA

- Memoria dinámica.
- Uso de la memoria dinámica.
- Concepto de memoria dinámica.
- Reserva y liberación de memoria dinámica en tiempo de ejecución.
- Utilización de matrices dinámicas.

Memoria dinámica

- Se crea en tiempo de ejecución.
- Las variables hasta ahora definidas se crean en tiempo de compilación.
- Un programa puede crear y liberar memoria en cualquier momento de la ejecución.
- Es responsabilidad del programador liberar la memoria dinámica utilizada.

Esquema del mapa de la memoria



Uso de la memoria dinámica

- En C las funciones `malloc()`, `calloc()`, `realloc()` y `free()` asignan y liberan memoria de un bloque denominado el montículo del sistema.
- Definidas en `stdlib.h`.
- Las funciones `malloc()`, `calloc()`, `realloc()` asignan memoria utilizando asignación dinámica debido a que puede gestionar la memoria durante la ejecución de un programa.
- Estas funciones requieren generalmente un moldeado (conversión de tipos).

Concepto de memoria dinámica

- La memoria dinámica es un espacio de almacenamiento que se solicita en tiempo de ejecución.
- A medida que el proceso va necesitando espacio para más datos, va solicitando más memoria al sistema operativo para guardarlos.
- El medio para manejar la memoria que otorga el sistema operativo, es el puntero, puesto que no podemos saber en tiempo de compilación dónde nos dará huecos el sistema operativo (en la memoria de nuestro PC).

Reserva y liberación de memoria dinámica en tiempo de ejecución

- Función malloc(): Reserva espacio en la memoria dinámica.
- Prototipo: void *malloc(size_t n);
- Devuelve un puntero a void, tenemos que utilizar un casting o moldeo al tipo de datos deseado.
- Uso:
 tipo *puntero;
 puntero = (tipo *)malloc(tamaño en bytes);
- En caso de que no haya suficiente memoria, malloc devuelve NULL.

Reserva y liberación de memoria dinámica en tiempo de ejecución

- Se aconseja el uso de `sizeof()` para reservar memoria.
- Ejemplo:
 `float *bloqueMEM;`
 `bloqueMEM = (float *)malloc(100*sizeof(float));`
- En caso de reservar memoria para cadenas de caracteres, reservar 1 char de mas para contener el `'\0'`.

Ejemplo con char *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void main(){
    char cad[121], *ptr;
    int lon;
    puts("Introduce una línea de texto:");
    gets(cad);
    lon=strlen(cad);
    // Reserve memoria para la longitud de la cadena+1.
    ptr = (char *)malloc((lon+1)*sizeof(char));
    strcpy(ptr, cad);
    printf("ptr = %s", ptr);
    free(ptr);
}
```

Utilización de matrices dinámicas

- Para definir matrices dinámicas, tenemos que utilizar punteros del tipo que queramos los elementos.

```
int *numeros, n, i;
```

```
printf("Introduzca el número de elementos:");
```

```
scanf("%d",&n);
```

```
numeros = (int *)malloc(n * sizeof(int));
```

```
for (i = 0 ; i < n ; i++){
```

```
    printf("Introduzca número %d: ", i);
```

```
    scanf("%d", &numeros[i]);
```

```
}
```

Uso de malloc() para array de mas dimensiones

// Reservar memoria para una tabla de 5 x 5 elementos.

```
int **tabla, i;
```

// Sitio para las 5 filas:

```
tabla = (int **)malloc(5 * sizeof(int *));
```

// Por cada fila reservamos para las 5 columnas.

```
for (i = 0 ; i < 5 ; i++ ){
```

```
    tabla[i] = (int *)malloc(5 * sizeof(int));
```

```
}
```

Liberación de la memoria

- Para liberar la memoria ocupada por malloc() utilizaremos la función free().
- Prototipo:
 - void free(void *);
- Ejemplo:

```
int *ad; // Reserva para un entero.  
ad = (int *)malloc(sizeof(int));  
char *s; // Reserva para 100 char.  
s = (char *)malloc(100 * sizeof(char));  
  
// Liberar ambos punteros:  
free(ad);  
free(s);
```

Función calloc()

- Reserva memoria dinámica para un número de elementos.
- Devuelve un puntero a void que tenemos que convertir.
- Si no hay suficiente memoria devuelve NULL.
- La forma de utilizarla:
 - puntero = calloc(número elementos, tamaño)
- Prototipo:
 - void *calloc(size_t n, size_t t);

Ejemplo de calloc()

```
#define N 5  
double *p;  
p = (double *)calloc(N, sizeof(double));  
  
char *c, B[121];  
puts("Introduce una cadena:");  
gets(B);  
c = (char *)calloc(strlen(B)+1, sizeof(char));  
strcpy(c, B);
```

Función realloc()

- Al igual que las funciones malloc() y realloc() permite reservar memoria dinámica.
- realloc() permite ampliar el bloque de memoria inicialmente reservado.
- Devuelve un void *.
- Uso:
 - tipo *puntero;
 - puntero=(tipo *)realloc(puntero a bloque, tamaño total del nuevo bloque);

Ejemplo realloc()

- Reserva memoria para una cadena y a continuación reserva memoria para otra mas larga.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(){
    char *cadena;
    int tam;
    tam = (strlen("Primavera")+1)*sizeof(char);
    cadena = (char *)malloc(tam);
    strcpy(cadena, "Primavera");
    puts(cadena);

    tam+=(strlen(" en Lupiana\n")+1)*sizeof(char);
    cadena = (char *)realloc(cadena, tam);
    strcat(cadena, " en Lupiana\n");
    puts(cadena);
    free(cadena);
    return 0;
}
```

Comparativa de las 3 funciones

Función	Finalidad	Inicialización	Parámetros	Ventaja principal	Riesgo principal
malloc()	Crear un bloque nuevo	No inicializa	Número total de bytes	Sencilla y evita inicialización innecesaria	Leer antes de inicializar produce valores indeterminados
calloc()	Crear un array	Inicializa todos los bytes a cero	Número de elementos y tamaño individual	Inicialización automática y detección del desbordamiento de la multiplicación	Puede inicializar memoria que después se sobrescribirá
realloc()	Redimensionar un bloque existente	Conserva los datos anteriores; la ampliación no se inicializa	Puntero anterior y nuevo tamaño total	Permite ampliar o reducir un bloque conservando contenido	Puede mover el bloque y devolver una dirección diferente

Ventajas / Inconvenientes malloc

- **Ventajas**
 - API sencilla.
 - Apropiada cuando el programa inicializará inmediatamente todo el bloque.
 - Permite expresar el tamaño utilizando el tipo apuntado:
 - `malloc(count * sizeof(*values));`
- **Inconvenientes:**
 - La memoria no está inicializada.
 - Debe comprobarse si devuelve NULL.
 - La expresión `count * sizeof(*values)` puede desbordarse si `count` procede de una fuente no fiable. A diferencia de `calloc()`, `malloc()` no detecta automáticamente ese desbordamiento.

Ventajas / Inconvenientes calloc

- **Ventajas**

- Resulta natural para arrays.
- Inicializa todos los bytes del bloque a cero.
- Detecta el desbordamiento de $\text{count} * \text{size}$ y falla en lugar de reservar un bloque demasiado pequeño.

- **Inconvenientes:**

- La inicialización puede ser innecesaria si el programa sobrescribe inmediatamente todo el bloque.
- “Todos los bytes a cero” no debe interpretarse como una inicialización semántica universal para cualquier representación posible. Para estructuras complejas, conviene inicializar explícitamente sus campos.

Ventajas / Inconvenientes realloc

- **Ventajas:**

- Amplía o reduce bloques existentes.
- Conserva el contenido hasta el mínimo entre el tamaño anterior y el nuevo.
- Si puede, redimensiona el bloque en su posición actual.
- Si recibe NULL, se comporta como una asignación nueva equivalente a malloc(size)

- **Inconvenientes:**

- Puede mover el bloque y devolver una dirección diferente.
- Si amplía el bloque, la zona nueva queda sin inicializar.
- Todos los punteros que apuntaban al bloque anterior quedan invalidados si este se mueve.
- Asignar directamente el resultado al puntero original puede provocar una fuga: ***Si falla, devuelve NULL, pero el bloque original permanece asignado y se pierde su dirección***

realloc patrón correcto

```
int *temporal = realloc(values, new_size);  
if (temporal != NULL) {  
    values = temporal;  
}
```

Memory leaks / dangling pointers

- **Memory leak (fuga de memoria):** la memoria continúa reservada, pero el programa ha perdido la posibilidad de liberarla.
 - Laguna de memoria sin liberar.
- **Dangling pointer (puntero colgante):** el puntero continúa existiendo, pero la memoria a la que apuntaba ya no es válida.

Ejemplo: Memory leaks

```
void process(void){  
    int *values = malloc(100 * sizeof(*values));  
    if (values == NULL) {  
        return;  
    }  
    values[0] = 42;  
    /* Falta free(values) */  
}
```

Al terminar process(), desaparece la variable values, pero el bloque reservado continúa ocupado. Ya no existe ningún puntero que permita liberarlo

Ejemplo 2: Memory leaks

- `int *values = malloc(100 * sizeof(*values));`
- `values = malloc(200 * sizeof(*values));`
- La segunda asignación sustituye la dirección anterior. El primer bloque queda inaccesible.

Ejemplo 3: Memory leaks

- `values = realloc(values, new_size);`
- Si `realloc()` falla, devuelve `NULL`, pero el bloque original continúa reservado. Al sobrescribir `values`, se pierde su dirección.
- Utilizar un temporal como antes y comprobar si ha devuelto `NULL`.

Fugas por rutas de error: Memory leaks

```
int process(void){
    char *buffer = malloc(1024);
    if (buffer == NULL) {
        return -1;
    }
    FILE *file = fopen("data.bin", "rb");
    if (file == NULL) {
        return -1; /* Fuga de buffer */
    }
    free(buffer);
    fclose(file);
    return 0;
}
```

- **Corrección:**
if (file == NULL) {
 free(buffer);
 return -1;
}

Dangling Pointers

- Un dangling pointer, o puntero colgante, apunta a una zona de memoria cuya vida útil ha terminado.

```
int *value = malloc(sizeof(*value));  
if (value != NULL) {  
    *value = 42;  
    free(value);  
    printf("%d\n", *value); /* Error: use-after-free */  
}
```

Después de free():

- value conserva una dirección;
- esa dirección ya no pertenece al programa;
- acceder mediante el puntero produce comportamiento indefinido.

Dangling Pointers

- Para prevenir problemas como el ejemplo anterior:
- Siempre que liberemos un puntero luego ponerlo a NULL
 - `free(value);`
 - `value = NULL;`

Múltiples punteros al mismo bloque

```
int *first = malloc(sizeof(*first));
```

```
int *second = first;
```

```
free(first);
```

```
first = NULL;
```

```
*second = 10; /* second sigue siendo dangling */
```

Devolver la dirección de una variable local

- Una variable local automática deja de existir cuando termina su bloque. El puntero devuelto queda colgante.

```
int *get_value(void){  
    int value = 42;  
    return &value; /* Dirección inválida al terminar la función */  
}
```

Posibles Soluciones

```
int get_value(void){  
    return 42;  
}
```

```
void get_value(int *result){  
    if (result != NULL) {  
        *result = 42;  
    }  
}
```

Puntero a una zona redimensionada

```
int *values = malloc(10 * sizeof(*values));  
int *element = &values[5];  
int *temporary = realloc(values, 20 * sizeof(*values));  
if (temporary != NULL) {  
    values = temporary;  
}  
printf("%d\n", *element); /* Puede ser dangling */
```

- Al hacer realloc la memoria se puede reasignar y el puntero puede cambiar de dirección.

Soluciones

```
size_t index = 5;  
int *temporary = realloc(values, 20 * sizeof(*values));  
if (temporary != NULL) {  
    values = temporary;  
    int *element = &values[index];  
}
```

- Se reconstruye el puntero.

Doble free

- Otro posible error con los punteros. Intentar liberar el mismo bloque de memoria.

```
int *value = malloc(sizeof(*value));
```

```
free(value);
```

```
free(value); /* Comportamiento indefinido */
```

- **Solución:**

```
free(value);
```

```
value = NULL;
```

```
free(value); /* Es seguro: free(NULL) no hace nada */
```

Estrategias de Prevención

- Definir la propiedad de la memoria:
 - Cada bloque debe tener un propietario claramente definido:

Quien reserva

|

+-- conserva la propiedad

|

+-- o la transfiere explícitamente

La documentación de una función debe aclarar si:

- devuelve memoria que el llamador debe liberar;
- recibe únicamente una referencia prestada;
- toma la propiedad del puntero recibido.

Ejemplo

- **char *create_message(void);**
 - La función devuelve memoria dinámica.
 - El llamador debe ejecutar free().

Una única zona de limpieza

- Cuando una función maneja varios recursos, centralizar su liberación reduce olvidos:

cleanup:

```
free(buffer);  
free(records);  
fclose(file);
```

```
Invalidar punteros después de liberar:  
free(buffer);  
buffer = NULL;
```

- **free(NULL) es válido**, por lo que inicializar punteros a NULL simplifica este patrón.

Evitar alias innecesarios

- Cuantos más punteros apunten al mismo bloque, más difícil resulta controlar su vida útil:

```
int *owner = malloc(...);  
int *alias1 = owner;  
int *alias2 = owner;
```
- alias1 y alias2 son referencias prestadas y no pueden utilizarse después de que el propietario libere el bloque.

Liberar estructuras de forma simétrica

- Si una estructura contiene memoria dinámica:

```
typedef struct {  
    char *name;  
    int *values;  
} record_t;
```

- ***Liberar en una función.*** }

```
void record_destroy(record_t *record){  
    if (record == NULL) {  
        return;  
    }  
    free(record->name);  
    free(record->values);  
    record->name = NULL;  
    record->values = NULL;
```

Liberar estructuras de forma simétrica

- Si la propia estructura también fue creada dinámicamente.

```
void record_destroy(record_t *record){  
    if (record == NULL) {  
        return;  
    }  
    free(record->name);  
    free(record->values);  
    free(record);  
}
```

Detección

- Durante la compilación pueden activarse advertencias:
 - **gcc** -std=c11 -Wall -Wextra -Wpedantic -g programa.c -o programa
- Con **valgrind**:
 - valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes \
./programa

Detección

- Los sanitizers también permiten detectar accesos inválidos durante la ejecución.

```
gcc -std=c11 -g \  
    -fsanitize=address,undefined \  
    -fno-omit-frame-pointer \  
    programa.c -o programa
```

```
./programa
```

Resumen

Problema	Situación	Consecuencia	Prevención
Memory leak	Se pierde el último puntero a memoria reservada	Consumo creciente de memoria	Liberación simétrica y rutas comunes de limpieza
Dangling pointer	Se utiliza un puntero después de terminar la vida del objeto	Comportamiento indefinido	Control de propiedad y vida útil
Use-after-free	Se accede tras ejecutar free()	Lecturas inválidas, corrupción o fallo	No reutilizar alias y anular punteros
Double free	Se libera dos veces el mismo bloque	Corrupción del heap	Propietario único y asignación a NULL
Fuga con realloc()	Se sobrescribe el puntero original al fallar	Pérdida del bloque anterior	Utilizar un puntero temporal

Fragmentación y estrategias de asignación

- La fragmentación aparece cuando la memoria libre existe, pero su distribución o el coste interno de administrarla impiden aprovecharla eficientemente.
- Puede ser externa o interna

Fragmentación externa

- La memoria libre queda dividida en bloques pequeños separados por bloques ocupados.
 - [Ocupado][Libre 32 B][Ocupado][Libre 64 B][Ocupado][Libre 16 B]
- Aunque existen 112 bytes libres, no hay un bloque contiguo de 100 bytes:
 - Una asignación grande podría fallar o provocar que el asignador solicite más memoria al sistema.

Ejemplo

- **Reservamos 3 bloques:**

```
void *a = malloc(100);
```

```
void *b = malloc(200);
```

```
void *c = malloc(100);
```

```
free(b);
```

- **Quedaría así:**

[a: 100 bytes ocupados]

[b: 200 bytes libres]

[c: 100 bytes ocupados]

- Si posteriormente se solicita un bloque mayor que 200 bytes, el hueco central no será suficiente

Fragmentación interna

- El asignador puede reservar más memoria que la solicitada debido a:
 - alineamiento;
 - metadatos internos;
 - tamaños mínimos de bloque;
 - redondeo de tamaños.
- Ejemplo: `void *ptr = malloc(13);`
- Aunque se soliciten 13 bytes, el bloque real puede ocupar más memoria. La diferencia no puede utilizarla la aplicación directamente.
 - ***No tiene nada que ver con el padding de una estructura.***

Reutilización y fusión de bloques

- En Ubuntu/Lubuntu, el asignador de **glibc** **administra** la **memoria** mediante bloques denominados **chunks**.
- Cuando se liberan bloques vecinos, puede fusionarlos para formar un bloque libre mayor. glibc también utiliza múltiples áreas denominadas *arenas* para reducir la contención entre hilos.
 - **Antes:**
 - [Libre 64 B][Libre 128 B]
 - **Después de fusionar:**
 - [Libre 192 B]

Estrategias para reducir la fragmentación

- Reducir asignaciones pequeñas independientes:

- En lugar de reservar cada elemento por separado:

```
for (size_t i = 0; i < count; ++i) {  
    elements[i] = malloc(sizeof(element_t));  
}
```

- menos llamadas a malloc();
- menos metadatos internos;
- una única llamada a free();
- datos contiguos y código de limpieza más simple.

- Puede reservarse un array contiguo:

- `element_t *elements = malloc(count * sizeof(*elements));`

Reservar capacidad con margen

- En vectores dinámicos, ampliar un elemento cada vez genera múltiples reasignaciones.
- No es una buena práctica hacer:
 - `items = realloc(items, (count + 1) * sizeof(*items));`
- Es preferible manejar por separado el tamaño utilizado y la capacidad.

Ejemplo

```
if (count == capacity) {  
    size_t new_capacity = capacity == 0 ? 8 : capacity * 2;  
    item_t *temporary = realloc( items, new_capacity * sizeof(*items) );  
  
    if (temporary == NULL) {  
        /* Gestionar el error */  
    } else {  
        items = temporary;  
        capacity = new_capacity;  
    }  
}
```

Esto reduce la frecuencia de llamadas a realloc() y evita redimensionar por cada elemento.

Reutilizar el mismo buffer

```
uint8_t *buffer = malloc(BUFFER_SIZE);
if (buffer == NULL) {
    return -1;
}
while (running) {
    process_data(buffer, BUFFER_SIZE);
}
free(buffer);
```

// Más consumo de memoria:

```
while (running) {
    uint8_t *buffer = malloc(BUFFER_SIZE);
    if (buffer == NULL) {
        break;
    }
    process_data(buffer, BUFFER_SIZE);
    free(buffer);
}
```

Pools de memoria

- Un pool reserva inicialmente una región de memoria y la divide en bloques de tamaño fijo:
- Pool de memoria
 - | Bloque 0 | Bloque 1 | Bloque 2 | Bloque 3 |

Pools de memoria

- En lugar de llamar repetidamente a **malloc()** y **free()**, la aplicación obtiene un bloque libre del pool y lo devuelve cuando termina de utilizarlo.
- **Resulta apropiado cuando:**
 - Los objetos tienen el mismo tamaño.
 - Se crean y destruyen frecuentemente.
 - Se quiere reducir el número de asignaciones.
 - Se necesita un tiempo de asignación más predecible.

Ejemplo

```
#define POOL_SIZE 100
```

```
typedef struct {  
    int id;  
    char data[64];  
} object_t;
```

```
typedef struct {  
    object_t objects[POOL_SIZE];  
    unsigned char used[POOL_SIZE];  
} object_pool_t;
```

- **Inicialización:**

```
void pool_init(object_pool_t *pool){  
    memset(pool->used, 0, sizeof(pool->used));  
}
```

Ejemplo 2ª parte

- Obtención elemento:

```
object_t *pool_acquire(object_pool_t *pool)
{
    for (size_t i = 0; i < POOL_SIZE; ++i) {
        if (pool->used[i] == 0) {
            pool->used[i] = 1;
            return &pool->objects[i];
        }
    }
    return NULL;
}
```

- Liberación:

```
void pool_release(object_pool_t *pool, object_t *object)
{
    ptrdiff_t index = object - pool->objects;
    if (index >= 0 & index < POOL_SIZE) {
        pool->used[index] = 0;
    }
}
```

Ventajas / Inconvenientes

Ventajas:

- Reduce llamadas al asignador general.
- Evita fragmentación externa entre objetos del pool.
- Permite limitar previamente el consumo máximo.
- Mejora la localidad de caché.

Inconvenientes:

- Capacidad inicialmente limitada.
- Puede desperdiciar memoria si apenas se utiliza.
- Solo resulta sencillo cuando los objetos tienen tamaño uniforme.
- Requiere lógica adicional para gestionar bloques libres.

Asignación mediante arenas

- Una arena reserva una región grande y entrega fragmentos consecutivos de ella.
- **Arena**
 - | Objeto A | Objeto B | Buffer C | Espacio libre |
- Una implementación sencilla mantiene:
 - La dirección inicial.
 - La capacidad total.
 - El desplazamiento utilizado.

```
typedef struct {  
    uint8_t *memory;  
    size_t capacity;  
    size_t offset;  
} arena_t;
```

Asignación mediante arenas

- **Asignación:**

```
void *arena_allocate(arena_t *arena, size_t size){  
    if (size > arena->capacity - arena->offset) {  
        return NULL;  
    }  
    void *result = arena->memory + arena->offset;  
    arena->offset += size;  
    return result;  
}
```

Liberación:

```
void arena_reset(arena_t *arena) {  
    arena->offset = 0;  
}
```

Asignación mediante arenas

- La memoria no se libera objeto por objeto. Al terminar la operación, se reinicia o destruye toda la arena.
- Es útil cuando varios objetos tienen la misma vida útil:
 - Datos temporales de una petición.
 - Árboles sintácticos.
 - Resultados intermedios.
 - Procesamiento de mensajes de red.

Ventajas:

- Asignaciones muy rápidas.
- Liberación conjunta.
- Pocos metadatos.
- Menor riesgo de olvidar liberaciones individuales.

Inconvenientes:

- No permite liberar fácilmente un objeto aislado.
- Puede retener memoria hasta que finalice toda la operación.
- Debe gestionarse correctamente el alineamiento.

Separar objetos según tamaño y vida útil

- Mezclar objetos temporales, permanentes, pequeños y grandes puede aumentar la fragmentación.
- **Patrón problemático:**
 - [pequeño permanente]
 - [grande temporal]
 - [pequeño permanente]
 - [grande temporal]

Cuando los bloques grandes se liberan
Quedan huecos entre objetos permanentes:

[pequeño permanente]
[hueco grande]
[pequeño permanente]
[hueco grande]

Esto puede implementarse mediante:

- Pools para objetos uniformes.
- Arenas para objetos temporales.
- Buffers reutilizables para operaciones repetitivas.
- Asignaciones independientes para bloques grandes.

Asignaciones grandes en Linux

- El asignador de **glibc** gestiona habitualmente bloques pequeños y medianos mediante áreas denominadas arenas.
- Para bloques grandes puede utilizar **mmap()**, permitiendo que la memoria se devuelva directamente al sistema al liberarla.
- El umbral usado para decidirlo es dinámico.

Resumen

Estrategia	Uso principal	Ventaja	Inconveniente
Buffer reutilizable	Operaciones repetitivas	Evita asignaciones continuas	Retiene memoria
Pool	Objetos de tamaño fijo	Asignación rápida y predecible	Capacidad limitada
Arena	Objetos con vida útil común	Liberación conjunta	Difícil liberar individualmente
Array contiguo	Colecciones homogéneas	Buena localidad de caché	Puede requerir realloc()
Asignador general	Uso genérico	Flexible y sencillo	Menor control sobre fragmentación

Valgrind y Sanitizers: Análisis dinámico

- El análisis dinámico examina el programa mientras se ejecuta.
- Solo puede detectar errores presentes en las rutas de código ejecutadas durante la prueba, por lo que debe combinarse con casos de prueba representativos.

- Programa con: desbordamiento del bloque, lectura después de free() y doble liberación:

```
#include <stdlib.h>

int main(void){
    int *values = malloc(4 * sizeof(int));
    if (values == NULL) {
        return EXIT_FAILURE;
    }
    values[4] = 100;
    free(values);
    values[0] = 200;
    free(values);
    return EXIT_SUCCESS;
}
```

Valgrind Memcheck

- **Valgrind Memcheck** ejecuta el binario en un entorno instrumentado.
- Puede detectar accesos fuera de límites, uso de memoria liberada, valores no inicializados, liberaciones incorrectas y fugas de memoria.
- **Instalación:**
 - `sudo apt update`
 - `sudo apt install valgrind`

Compilación

```
gcc -std=c11 \  
-Wall -Wextra -Wpedantic \  
-g -O0 \  
memoria.c \  
-o memoria
```

Para ejecutar:

```
valgrind ./memoria
```

- **-g**: añade símbolos de depuración.
- **-O0**: facilita relacionar el informe con el código fuente.

Análisis Completo

**valgrind **

**--tool=memcheck **

**--leak-check=full **

**--show-leak-kinds=all **

**--track-origins=yes **

./memoria

--leak-check=full muestra detalles de fugas;
--show-leak-kinds=all incluye todas sus categorías, y
--track-origins=yes ayuda a localizar el origen de valores no inicializados.

Mensajes habituales

- **Escritura inválida: Invalid write of size 4**
 - Indica que se ha escrito fuera de una región válida o en memoria ya liberada.
- **Lectura inválida: Invalid read of size 4**
 - Puede indicar:
 - lectura fuera de límites;
 - uso después de free();
 - puntero inválido.

Mensajes habituales II

- **Liberación incorrecta: Invalid free()**
 - Puede deberse a:
 - doble liberación;
 - liberar un puntero interior;
 - liberar memoria no obtenida dinámicamente.
- **Valor no inicializado: Use of uninitialised value**
 - Indica que se utiliza un dato antes de asignarle un valor definido.

Categoría de las fugas

Categoría	Significado
definitely lost	No queda ningún puntero al bloque
indirectly lost	El bloque dependía de otro bloque perdido
possibly lost	Solo queda un puntero que podría apuntar al interior
still reachable	Queda algún puntero válido, pero la memoria no fue liberada

AddressSanitizer

- **AddressSanitizer, ASan**, instrumenta el programa durante la compilación.
- Requiere recompilar, pero normalmente ejecuta las pruebas más rápidamente que Valgrind.

```
gcc -std=c11 \  
    -Wall -Wextra -Wpedantic \  
    -g -O1 \  
    -fsanitize=address \  
    -fno-omit-frame-pointer \  
    memoria.c \  
    -o memoria_asan
```

- **Para ejecutar: ./memoria_asan**

AddressSanitizer

- Puede detectar, entre otros:
 - heap-buffer-overflow;
 - stack-buffer-overflow;
 - use-after-free;
 - doble liberación;
 - determinadas fugas de memoria.
- **ASan** suele detener el programa en el primer error grave. Tras corregirlo, debe ejecutarse de nuevo para descubrir los siguientes.

UndefinedBehaviorSanitizer

- **UBSan** detecta algunas operaciones cuyo comportamiento no está definido por C.

```
gcc -std=c11 \  
    -Wall -Wextra -Wpedantic \  
    -g -O1 \  
    -fsanitize=undefined \  
    -fno-omit-frame-pointer \  
    programa.c \  
    -o programa_ubsan
```

Puede ayudar a detectar:

- desbordamientos aritméticos con signo;
- desplazamientos de bits inválidos;
- divisiones enteras por cero;
- determinados accesos desalineados;
- conversiones o índices inválidos.

Asan y UBSan combinados

- Se pueden combinar ambos:

```
gcc -std=c11 \  
    -Wall -Wextra -Wpedantic \  
    -g -O1 \  
    -fsanitize=address,undefined \  
    -fno-omit-frame-pointer \  
    programa.c \  
    -o programa_debug
```

- Ejecución: ./programa_debug

LeakSanitizer

- **LeakSanitizer** está orientado a detectar memoria reservada que no se libera.

```
gcc -std=c11 \  
-g -O1 \  
-fsanitize=leak \  
programa.c \  
-o programa_lsan
```

Puede combinarse con ASan:

```
gcc -std=c11 \  
-g -O1 \  
-fsanitize=address,undefined \  
programa.c \  
-o programa_sanitized
```

Comparativa

Característica	Valgrind Memcheck	Sanitizers
Requiere recompilar	No necesariamente	Sí
Velocidad	Más lento	Generalmente más rápido
Símbolos -g	Muy recomendables	Muy recomendables
Fugas	Sí	LeakSanitizer
Memoria no inicializada	Muy útil	ASan no es su herramienta principal
Desbordamientos	Sí	ASan
Comportamiento indefinido	Limitado	UBSan
Uso habitual	Análisis detallado y segunda comprobación	Desarrollo, pruebas y CI

Flujo Recomendado

Compile con warnings

|

v

Ejecutar ASan y UBSan

|

v

Corregir todos los errores

|

v

Ejecutar Valgrind Memcheck

|

v

Comprobar fugas y accesos inválidos

Compilación en desarrollo

```
gcc -std=c11 \  
-Wall -Wextra -Wpedantic \  
-Wconversion \  
-g -O1 \  
-fsanitize=address,undefined \  
-fno-omit-frame-pointer \  
programa.c \  
-o programa
```

Resumen

- **Valgrind** analiza el binario durante su ejecución y ofrece informes detallados.
- **ASan** detecta principalmente errores de acceso y vida útil de memoria.
- **UBSan** detecta determinadas operaciones con comportamiento indefinido.
- **LeakSanitizer** identifica memoria dinámica no liberada.
- Ninguna herramienta sustituye una buena gestión de propiedad ni unas pruebas completas.
- Un análisis limpio solo demuestra que no aparecieron errores en las rutas ejecutadas.